JAVA • DATA STRUCTURES • GRAPH ALGORITHMS

Emergency Route Optimizer

Academic Project Report

Design, Implementation, Testing and Prototype Evaluation

A graph-based prototype for estimating fast routes and comparing alternatives

---

Student name:	[Enter student name]
Roll / registration no.:	[Enter roll number]
Programme / semester:	[Enter programme and semester]
Department / institution:	[Enter department and institution]
Project supervisor:	[Enter supervisor name]
Implementation:	Java 17+, Maven, Spring Boot
Core algorithms:	Dijkstra and Yen's K-shortest loopless paths
Report date:	31 October 2026

Safety and scope notice. This is an academic prototype and decision-support demonstration. It is not a certified emergency-dispatch system. The in-house graph algorithms operate on a fictional sample graph; the web route endpoint delegates real-road route calculations to OSRM's standard driving profile. The prototype does not provide validated emergency-vehicle access rules, live traffic, verified closures, traffic-signal priority, or operational guarantees. It must not be used to direct an actual emergency response.

Contents

1 Abstract **2**

2 Introduction and Problem Statement **2**

3 Objectives **2**

4 Scope, Assumptions and Functional Requirements **3**
4.1 Included in the academic prototype . . . 3
4.2 Important assumptions and boundaries . . . 3

5 System Architecture **3**
5.1 Backend endpoints . . . 4
5.2 External service roles . . . 4

6 Graph Model and Travel-Time Formula **4**

7 Algorithms and Data Structures **5**
7.1 Adjacency-list graph . . . 5
7.2 Dijkstra's shortest path . . . 5
7.3 Yen's K-shortest loopless paths . . . 5
7.4 Validation and deterministic behavior . . . 5

8 Implementation **6**

9 Testing and Verification **6**
9.1 Automated checks . . . 6
9.2 Observed web demonstration . . . 6
9.3 Manual acceptance tests . . . 7

10 Results **7**
10.1 Fictional DSA sample . . . 7
10.2 Real-road web example . . . 7

11 Known Limitations and Risk Boundary **8**

12 Future Work **8**

13 Conclusion **9**

A Project File Map **9**

B Final Submission Notes **10**

1 Abstract

The Emergency Route Optimizer is an academic prototype that demonstrates graph-based route planning and a web-integrated mapping workflow. Its in-house Java engine models a road network as a directed weighted graph. Dijkstra's algorithm computes a minimum-estimated-time path, while Yen's algorithm generates distinct loopless alternatives. The sample engine supports directional road closures, traffic-multiplier updates, input validation, and unreachable-destination handling.

A separate Spring Boot web backend exposes health, route, and place-search endpoints. The browser interface uses HTML, CSS, JavaScript, and Leaflet to display locations and provider-returned route geometry. For the real-road demonstration, coordinates are sent to OSRM's standard driving profile; place names are resolved through an explicit Nominatim search. The web integration and the in-house DSA engine are separate paths: the current web route endpoint does not execute Dijkstra or Yen over an imported real-world city graph.

The dependency-free regression suite was expanded and passed 31 checks in the validation environment. A user-observed web demonstration returned an OSRM route of 6.7559 km and 573.8 seconds for a sample pair of Delhi coordinates. These results demonstrate functionality under the tested conditions, not operational safety or live-traffic accuracy.

Keywords: emergency route planning, directed graph, Dijkstra, Yen's algorithm, Spring Boot, OSRM, Nominatim, Leaflet, route alternatives.

2 Introduction and Problem Statement

Emergency response journeys may involve several possible road paths. A shorter road path is not necessarily the fastest if its estimated speed is lower, its traffic factor is higher, or a road segment is unavailable. A planning tool can help compare routes using explicit assumptions and can recalculate when a road is marked closed or a traffic multiplier changes.

The academic problem addressed here is to model road connectivity and estimate route cost in a reproducible way, then expose a separate map-backed demonstration that can request real-road route information from an external routing engine.

The project does not attempt to replace emergency dispatchers, grant special road privileges, control traffic signals, or certify the safety of a route. It is a prototype for demonstrating data structures, algorithms, API integration, and user-interface design.

3 Objectives

The project objectives are:

- represent locations and directional road segments using an adjacency-list graph;
- calculate estimated road travel time from distance, nominal speed, traffic multiplier, and fixed delay;
- find a minimum-estimated-time path using Dijkstra's algorithm;
- generate distinct loopless alternatives using Yen's K-shortest paths method;
- exclude closed road segments from subsequent route calculations;
- validate malformed input and handle unknown or unreachable locations;
- provide a Java REST backend and a map-based demonstration interface;

- document test evidence, assumptions, limitations, and future research work.

4 Scope, Assumptions and Functional Requirements

4.1 Included in the academic prototype

- A deterministic fictional network containing eight named locations and twenty-four directed road segments.
- Dijkstra shortest-time routing and up to ten requested Yen alternatives in the in-house engine.
- A console menu for route calculation, network display, road closure/reopening, traffic-factor changes, and bidirectional road addition.
- A Spring Boot backend with a health endpoint, a real-road routing endpoint, and a place-search endpoint in the working IntelliJ project.
- A Leaflet-based browser interface with origin/destination place search, optional browser geolocation, route geometry, distance/duration results, map-style selection, and multiple response-team selection.
- A Dispatch Workspace theme selector, incident-planning fields, and a route-readiness summary.

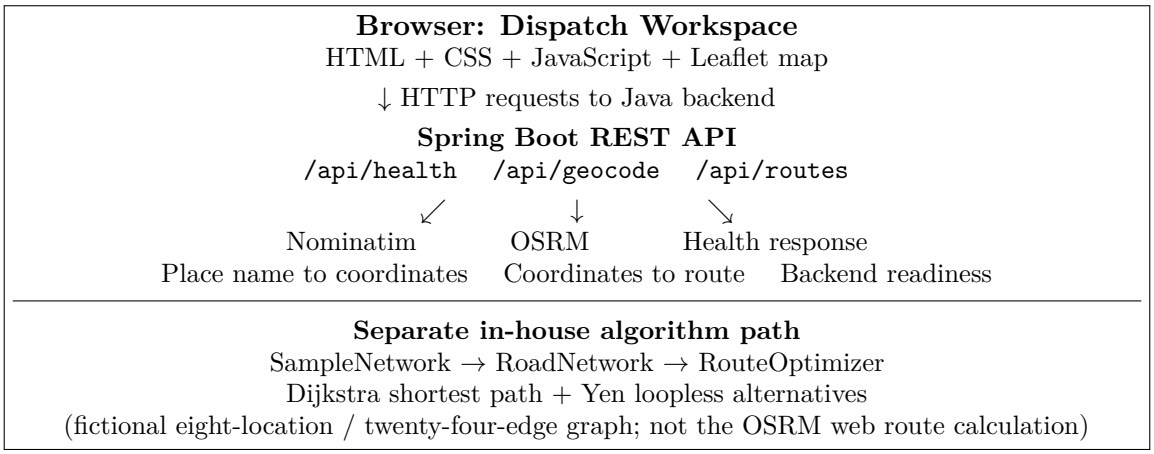
4.2 Important assumptions and boundaries

The in-house DSA graph is fictional and in memory. Its weights are configured sample values. The browser route endpoint calls OSRM's standard `driving` profile. The place-search endpoint uses Nominatim. Public map, geocoding, and routing services require internet access and may rate-limit or become unavailable.

The multiple-team selector records which response teams are selected for an incident in the interface. It does not track each vehicle's current position, create separate routes for each team, transmit dispatch orders, or communicate with emergency-service control rooms. Incident type, priority, and notes are planning fields; they do not alter the routing cost.

5 System Architecture

The working system contains a front end, a Spring Boot API layer, external geographic services, and a separate in-house DSA engine.



5.1 Backend endpoints

Endpoint	Purpose
`GET /api/health`	Returns a simple JSON readiness response so the interface can show whether the backend is responding.
`GET /api/geocode?q=...`	Performs an explicit text search and returns candidate place labels and coordinates. The UI requires a user action rather than sending one search request per keystroke.
`GET /api/routes`	Accepts origin/destination latitude and longitude query parameters, validates their ranges, calls the external routing service, and returns route JSON including distance, duration, and geometry where provided.

5.2 External service roles

OSRM: The route integration formats coordinates in longitude,latitude order, calls the `/route/v1/driving/` service, requests GeoJSON geometry, and asks for route alternatives. OSRM documentation notes that alternative routes can be requested, but an alternative is not guaranteed for every request [3].

Nominatim: The geocoding integration converts a user-entered place/address string into candidate place results. The working integration includes a descriptive User-Agent, caching of successful results, request spacing, and English preference. The public Nominatim service has a strict low-volume policy, including a maximum of one request per second and a requirement to identify the application [4].

Leaflet and map layers: Leaflet renders the interactive map and GeoJSON route. Available tile styles can include street, humanitarian, terrain, satellite, and cloudless layers. Label language, building footprints, road names, and water features depend on the data and rendering coverage in each area. The OpenStreetMap public tile service is best-effort, has no SLA, requires attribution, and has restrictions against bulk tile downloading [5,7].

6 Graph Model and Travel-Time Formula

Let $G = (V, E)$ be a directed graph. Each vertex represents a location; each directed edge represents a road segment. A two-way road is modelled as two independent directed edges, such as `R04-F` and `R04-R`, so the status of each direction can differ.

For edge e, the model stores distance d_e in kilometres, nominal speed v_e in kilometres per hour, traffic multiplier $m_e \geq 1$, additional delay $a_e \geq 0$ in minutes, a descriptive risk value, and an open/closed flag.

$$w(e) = \left(\frac{d_e}{v_e}\right) 60 m_e + a_e.$$

For a path P, estimated time is $T(P) = \sum_{e \in P} w(e)$. The objective is

$$P^* = \arg \min_{P \in \mathcal{P}(s,t)} T(P),$$

where $\mathcal{P}(s,t)$ is the set of directed paths from origin s to destination t using only open edges. The model uses non-negative weights, so Dijkstra's algorithm is suitable.

The traffic multiplier is a simplified scenario input, not a measured traffic prediction. The descriptive risk points are not part of the objective function and must not be interpreted as a validated safety rating.

7 Algorithms and Data Structures

7.1 Adjacency-list graph

`RoadNetwork` stores locations, roads, and outgoing edges in maps. Route searches iterate only over outgoing edges from the currently expanded location. Directional edges make one-way restrictions and direction-specific closures representable.

7.2 Dijkstra's shortest path

A priority queue selects the vertex with the smallest known tentative time. When an open outgoing edge produces a lower time to a neighbour, the algorithm updates the distance and predecessor road. Once the destination is reached, predecessor roads are traversed to reconstruct the path.

For a binary-heap priority queue, a conventional time bound is $O((|V| + |E|) \log |V|)$, with $O(|V| + |E|)$ graph/search storage.

7.3 Yen's K-shortest loopless paths

The first accepted path is produced by Dijkstra. Yen's method then considers each edge position in the most recently accepted path. It fixes the prefix as a root path, bans the next edge of accepted paths sharing that prefix, bans prior root vertices, and performs a spur shortest-path search. Root and spur paths are combined. Candidate route signatures prevent duplicates, and a priority queue selects the next lowest-cost candidate.

The implementation permits a requested count from one to ten. Yen's algorithm performs repeated shortest-path searches; its cost depends on graph size, route count, and network structure.

7.4 Validation and deterministic behavior

Identifiers are normalized using locale-independent uppercase rules. Road models validate finite positive distance/speed, traffic multipliers, non-negative delay, and a bounded descriptive risk value. The route model copies its location and road lists and exposes immutable results. Stable road ordering and queue tie-breaking improve reproducibility for the supplied fixture.

8 Implementation

Component	Responsibility
`Location`	Stable ID and display name for a graph vertex.
`Road`	Directional road segment with weight inputs, open/closed state, and estimated travel time.
`RoadNetwork`	Adjacency-list storage and graph validation.
`Route`	Immutable path result with locations, roads, ETA, distance, descriptive risk, and signature.
`RouteOptimizer`	Dijkstra shortest path and Yen alternative generation.
`SampleNetwork`	Deterministic fictional graph fixture.
`EmergencyRouteApp`	Original console menu and scenario controls.
`EmergencyRouteBackendApplication`	Spring Boot application entry point.
`HealthController`	Backend readiness endpoint.
`RouteController` / `OsrmRoutingService`	Coordinate validation and external OSRM route request.
`GeocodingController` / `NominatimGeocodingService`	Explicit place search and provider response handling.
`Browser interface`	Place search, team selection, map display, route selection, and theme control.

9 Testing and Verification

9.1 Automated checks

The dependency-free regression suite is executed by `scripts/verify.sh`. It compiles production sources using `javac -release 17` and executes 31 assertions/checks. The latest recorded run in the validation environment passed all 31 checks. The checks include:

- sample graph size, route existence, path endpoints, and node-edge consistency;
- route ordering, unique signatures, and loopless alternatives;
- closed-road exclusion and recalculation after road/traffic changes;
- same-origin and unreachable-network behavior;
- invalid location and alternative-count validation;
- direction-specific closures, locale-independent identifiers, and travel-time arithmetic;
- invalid road data, duplicate graph identifiers, and route-result immutability.

The JUnit suite was expanded to 13 test methods covering the core invariants and additional model-validation scenarios. The documentation preparation environment did not have Maven installed, so the updated JUnit suite was not independently executed in that environment. A final `mvn test` run must be completed on the student's IntelliJ/Maven installation and its green summary retained as submission evidence.

9.2 Observed web demonstration

The developer previously ran the backend health endpoint and observed JSON status `UP`. A real-road request between approximate coordinates near India Gate and Red Fort in Delhi returned `code: Ok`, a distance of 6,755.9 metres, duration of 573.8 seconds, and a GeoJSON `LineString`. This is

one observed external-service response, not a fixed test oracle. OSRM responses can vary with the provider's road data and availability.

Place search is implemented through the Nominatim integration. The final local acceptance test must confirm that `/api/geocode?q=Patiala` returns a JSON array or a controlled JSON error, and that the UI handles non-JSON HTML error pages without an unhandled parsing exception.

9.3 Manual acceptance tests

ID	Action	Expected observation
M01	Open `/api/health`.	JSON response with status `UP`.
M02	Request a real route through `/api/routes`.	Provider response with route code, distance, duration, and geometry.
M03	Search a place through `/api/geocode`.	JSON candidate list or controlled JSON error.
M04	Select origin and destination in the UI.	Selected labels/coordinates are displayed.
M05	Calculate route and choose alternatives.	Route geometry and metrics update on the map/sidebar.
M06	Select multiple response teams.	Multiple selections are visible; UI explains shared-route limitation.
M07	Toggle Dispatch Workspace theme.	Sidebar changes theme while map layers/controls remain unchanged.
M08	Switch map styles and zoom.	Tiles and attribution remain visible where provider coverage permits.

10 Results

10.1 Fictional DSA sample

For the default sample graph, one observed deterministic result set is:

Rank	Path	ETA	Distance	Risk points
1	Central Hospital → East Market → City Stadium → Emergency Site	10.82 min	7.50 km	3.0
2	Central Hospital → North Junction → City Stadium → Emergency Site	12.15 min	8.50 km	3.0
3	Central Hospital → North Junction → Riverside Bridge → Emergency Site	12.62 min	7.00 km	4.0

These figures are deterministic outputs of the fictional values in `SampleNetwork.java`. They are not real-city measurements. They illustrate that the fastest estimated route need not be the shortest by distance.

10.2 Real-road web example

The previously observed Delhi example returned approximately 6.76 km and 9.56 minutes. The OSRM response included route geometry and one route candidate for that request. An alternative request does not guarantee multiple alternatives; the OSRM API returns alternatives only when suitable alternatives exist [3].

11 Known Limitations and Risk Boundary

1. The in-house Dijkstra/Yen engine is tested on a small fictional graph; it is not currently operating on a full imported city road graph.
2. The web route endpoint delegates route calculation to OSRM's standard driving profile. It does not apply ambulance, fire-appliance, or police-specific permissions or vehicle dimensions.
3. There is no live traffic feed, verified dynamic road-closure feed, weather feed, construction feed, or traffic-signal priority integration.
4. Selecting multiple teams is a user-interface planning feature. Separate vehicle positions, per-vehicle routes, dispatch messages, acknowledgements, and fleet status are not implemented.
5. GPS is a browser-permission-based origin input, not continuous fleet tracking.
6. Route duration is an external provider estimate, not a guaranteed arrival time or calibrated emergency ETA.
7. Map detail and label language depend on provider coverage and geographic data quality. Missing buildings, rivers, or labels cannot be fixed solely by changing CSS or zoom.
8. Public routing, geocoding, and tile services have usage limits and no operational availability guarantee. The UI is a prototype and requires internet access for external data.
9. There is no authentication, role-based access, audit history, database-backed incident record, failover routing, monitoring, or safety certification.

12 Future Work

A real-world research phase should be developed in controlled stages:

1. Import a licensed and validated geospatial road graph with one-way rules, turn restrictions, access restrictions, bridges, and vehicle constraints.
2. Connect the real road graph to the in-house Dijkstra/Yen engine and benchmark results against a trusted routing provider.
3. Define ambulance, fire, and police vehicle profiles with verified jurisdiction-specific access policies.
4. Integrate time-stamped traffic, incidents, closures, construction, and weather data with source provenance, expiry, and stale-data warnings.
5. Support separate starting locations and route requests for each response team, plus dispatcher assignment and acknowledgement workflows.
6. Add database persistence, authentication, authorization, audit trails, observability, fault handling, backups, and privacy controls.
7. Evaluate with historical scenarios and domain experts. Any operational trial requires formal safety review, emergency-service authorization, and human dispatcher oversight.

13 Conclusion

The Emergency Route Optimizer demonstrates a complete academic workflow across graph modelling, shortest-path search, alternative-route generation, simulation of changing road conditions, a Java REST backend, external geocoding/routing integration, and a map-based planning interface. The core DSA implementation remains independently testable on a controlled sample graph, while the browser demonstration can request real-road route data from an external provider.

The main academic contribution is the implementation and verification of an explainable graph-routing engine and its integration into a wider prototype. The system is not yet a real emergency dispatch solution: its real-road routing is provider-based and standard-driving, while emergency-specific access, live data, separate fleet dispatch, and safety validation remain future work.

References

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959. DOI: 10.1007/BF01386390.

[2] J. Y. Yen, "Finding the K Shortest Loopless Paths in a Network," *Management Science*, vol. 17, no. 11, pp. 712–716, 1971. DOI: 10.1287/mnsc.17.11.712.

[3] Project OSRM, "OSRM HTTP API Documentation," Route service and alternative-route parameters. https://project-osrm.org/docs/v26.4.0/http.

[4] OpenStreetMap Foundation, "Nominatim Usage Policy." https://operations.osmfoundation.org/policies/nominatim/.

[5] OpenStreetMap Foundation, "Tile Usage Policy." https://operations.osmfoundation.org/policies/tiles/.

[6] Leaflet, "Leaflet API Reference." https://leafletjs.com/reference.html.

[7] OpenStreetMap, "Copyright and Licence." https://www.openstreetmap.org/copyright.

A Project File Map

```
src/main/java/com/emergencyroute/
  EmergencyRouteApp.java
  EmergencyRouteBackendApplication.java
  BrowserLauncher.java (optional)
  Location.java
  Road.java
  RoadNetwork.java
  Route.java
  RouteOptimizer.java
  SampleNetwork.java
  api/HealthController.java
  api/RouteController.java
  api/GeocodingController.java
  integration/OsrmRoutingService.java
  integration/NominatimGeocodingService.java
src/main/resources/static/index.html
src/test/java/com/emergencyroute/RouteOptimizerTest.java
tools/VerificationSuite.java
scripts/verify.sh
README.md
report.tex / report.pdf
```

B Final Submission Notes

Before handing in the project, enter the student/institution details on the title page, verify that the local IntelliJ project contains all backend and web files listed in Appendix A, run `mvn test` and `mvn package`, run `scripts/verify.sh`, and capture screenshots of the test results and the working web/API demonstration. Do not mark manual checks as complete until they have been reproduced on the final local project.